[10]
✓ 8s

```python
import numpy as np
import pandas as pd
from gensim.models import Word2Vec
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from nltk.corpus import movie_reviews
import nltk
```

[11]
✓ 0s

```python
nltk.download('movie_reviews')
```

```
[nltk_data] Downloading package movie_reviews to /root/nltk_data...
[nltk_data]   Unzipping corpora/movie_reviews.zip.
True
```

[12]
✓ 4s

```python
documents = [(list(movie_reviews.words(fileid)), category)
             for category in movie_reviews.categories()
             for fileid in movie_reviews.fileids(category)]
```

[13]
✓ 0s

```python
np.random.shuffle(documents)
```

[17]
✓ 0s

```python
reviews = [' '.join(doc) for doc, label in documents]
labels = [1 if label == 'pos' else 0 for doc, label in documents]


tokenized_reviews = [review.lower().split() for review in reviews]

X_train, X_test, y_train, y_test = train_test_split(tokenized_reviews, labels, test_size=0.2, random_state=42)
```

[18]
✓ 36s

```python
word2vec_model = Word2Vec(sentences=X_train, vector_size=100, window=5, min_count=1, workers=4)
word2vec_model.train(X_train, total_examples=len(X_train), epochs=10)
```

[18]
36s

```python
word2vec_model = Word2Vec(sentences=X_train, vector_size=100, window=5, min_count=1, workers=4)
word2vec_model.train(X_train, total_examples=len(X_train), epochs=10)


def get_average_vector(review, model, vector_size=100):
    vectors = [model.wv[word] for word in review if word in model.wv]
    if len(vectors) == 0:
        return np.zeros(vector_size)
    return np.mean(vectors, axis=0)


X_train_vectors = np.array([get_average_vector(review, word2vec_model) for review in X_train])
X_test_vectors = np.array([get_average_vector(review, word2vec_model) for review in X_test])
```

```
WARNING:gensim.models.word2vec:Effective 'alpha' higher than previous training cycles
```

[19]
4s

```python
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
rf_classifier.fit(X_train_vectors, y_train)


y_pred = rf_classifier.predict(X_test_vectors)

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f}")
```

```
Accuracy: 0.6650
```

one key advantage of using Word2Vec embeddings over TF-IDF for capturing the meaning of a text is that Word2Vec captures semantic relationships and similarities between words based on their context in the corpus, allowing it to understand synonyms and analogies, whereas TF-IDF only represents words as frequency-based weights without considering semantic meaning.